

# Manoj RAG Chatbot — Complete Step-by-Step Guide

By Manoj Nahak | Built with LangChain, Groq, MongoDB & Streamlit

---

## What is a RAG Chatbot?

**RAG = Retrieval Augmented Generation**

A RAG chatbot answers questions based on YOUR custom data (PDFs, documents) instead of just general knowledge.

Your PDF → MongoDB (stores knowledge) → User asks question → LLM answers from your data

---

## Tools & Their Importance

| Tool                       | Purpose                       | Why Important                       |
|----------------------------|-------------------------------|-------------------------------------|
| Python                     | Programming language          | Everything runs on Python           |
| VS Code                    | Code editor                   | Where you write and run your code   |
| Virtual Environment (venv) | Isolated Python environment   | Keeps project packages separate     |
| LangChain                  | AI framework                  | Connects LLM + embeddings + MongoDB |
| Groq LLM                   | Language model (GPT-OSS-120B) | Generates final answers             |
| HuggingFace Embeddings     | Converts text to vectors      | Enables semantic search             |
| MongoDB Atlas              | Cloud vector database         | Stores your custom knowledge        |
| Streamlit                  | Web UI framework              | Creates the chatbot interface       |
| Git                        | Version control               | Tracks code changes                 |
| GitHub                     | Code hosting                  | Stores your code online             |

---

## 📁 Final Project Structure

```
Manoj_chatbot/
├── .streamlit/
│   ├── secrets.toml      ← API keys (never upload to GitHub!)
│   └── config.toml
├── cb_venv/              ← Virtual environment
├── .env                  ← Local environment variables
├── .env.example          ← Template for others
├── .gitignore            ← Files to exclude from GitHub
├── personal_code.py      ← Backend: embeddings + MongoDB + LLM logic
├── streamlit_runner.py   ← Frontend: Streamlit chatbot UI
├── upload_data.py        ← One-time script: uploads PDF to MongoDB
├── Manoj_image.JPG       ← Your profile image
├── requirements.txt      ← All Python packages
└── README.md             ← Project documentation
```

---

## 🔄 Complete Flow

```
1. PDF → upload_data.py → chunks → embeddings → MongoDB
2. User types question → Streamlit UI
3. Question → HuggingFace embedding → vector
4. Vector → MongoDB $vectorSearch → top matching chunks
5. Chunks + Question → Groq LLM → Final Answer
6. Answer displayed in Streamlit UI
```

---

## ⚙️ PART 1 — ENVIRONMENT SETUP

### Step 1 — Install Python

- Download from **python.org**
- Check "Add Python to PATH" during install
- Verify: `python --version`

## Step 2 — Install VS Code

- Download from [code.visualstudio.com](https://code.visualstudio.com)
- Install Python extension inside VS Code

## Step 3 — Create Project Folder

```
bash
mkdir Manoj_chatbot
cd Manoj_chatbot
```

## Step 4 — Create Virtual Environment

```
bash
python -m venv cb_venv
```

## Step 5 — Activate Virtual Environment

```
bash
# Windows
cb_venv\Scripts\activate

# Mac/Linux
source cb_venv/bin/activate
```

■ You'll see `(cb_venv)` at the start of terminal — means it's active 

## Step 6 — Install Required Packages

```
bash
pip install langchain
pip install langchain-groq
pip install langchain-huggingface
pip install langchain-text-splitters
pip install langchain-community
pip install pymongo
pip install streamlit
pip install python-dotenv
pip install pypdf
pip install sentence-transformers
```

Or install all at once using requirements.txt:

```
bash

pip install -r requirements.txt
```

## 🔑 PART 2 — API KEYS SETUP

### Step 1 — Get Groq API Key

1. Go to **console.groq.com**
2. Sign up / Login
3. Click "API Keys" → "Create API Key"
4. Copy the key

### Step 2 — Get MongoDB Connection String

1. Go to **mongodb.com** → Login
2. Create a free cluster (M0)
3. Click **Connect** → **Drivers** → **Python**
4. Copy the connection string:

```
mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/
```

Use a simple password — only letters and numbers, no special characters!

### Step 3 — Create `.env` file

```
bash

groq_api=your_groq_api_key_here
mongo_db=mongodb+srv://manoj:yourpassword@cluster0.xxxxx.mongodb.net/
```

### Step 4 — Create `.streamlit/secrets.toml`

```
toml

groq_api = "your_groq_api_key_here"
mongo_db = "mongodb+srv://manoj:yourpassword@cluster0.xxxxx.mongodb.net/"
```

## Step 5 — Create `.env.example` (for GitHub)

```
bash

groq_api=your_groq_api_key
mongo_db=your_mongodb_connection_string
```

---

## PART 3 — MONGODB SETUP

### Step 1 — Create Free Cluster

1. MongoDB Atlas → **Create Cluster**
2. Choose **M0 Free**
3. Give cluster a name (cannot change later!)
4. Select region closest to you

### Step 2 — Add IP Address

1. **Network Access** → **Add IP Address**
2. Click "**Allow Access from Anywhere**" → `0.0.0.0/0`
3. Click **Confirm**

### Step 3 — Create Database User

1. **Database Access** → **Add New Database User**
2. Set username and simple password
3. Give **Atlas Admin** permissions
4. Click **Add User**

### Step 4 — Upload Your PDF Data

Create `upload_data.py`:

```
python
```

```
import os
from urllib.parse import quote_plus
from pymongo import MongoClient
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
from dotenv import load_dotenv

load_dotenv()

# Connect to MongoDB
username = "manoj"
password = quote_plus("yourpassword")
mongo_db = f"mongodb+srv://{username}:{password}@cluster0.xxxxx.mongodb.net/?ap

client = MongoClient(mongo_db, tls=True, tlsAllowInvalidCertificates=True)
collection = client["sample_mflix"]["personalpdf"]

# Load PDF
loader = PyPDFLoader("your_file.pdf") # ← your actual PDF name
documents = loader.load()

# Split into chunks
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(documents)

# Load embedding model
embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-

# Upload to MongoDB
print(f"Uploading {len(chunks)} chunks to MongoDB...")
for i, chunk in enumerate(chunks):
    embedding_vector = embedding.embed_query(chunk.page_content)
    collection.insert_one({
        "text": chunk.page_content,
        "embedding": embedding_vector,
        "source": chunk.metadata.get("source", ""),
        "page": chunk.metadata.get("page", 0)
    })
    print(f"Uploaded chunk {i+1}/{len(chunks)}")

print("✅ Done! All data uploaded to MongoDB.")
```

```
bash
```

```
python upload_data.py
```

## Step 5 — Create Vector Search Index

1. MongoDB Atlas → **Search & Vector Search**
2. Click "**Create Search Index**"
3. Select "**Bring your own embeddings**"
4. Select "**JSON Editor**"
5. Set:
  - Index Name: `vector_index`
  - Database: `sample_mflix`
  - Collection: `personalpdf`
6. Paste this JSON:

```
json
```

```
{
  "fields": [
    {
      "type": "vector",
      "path": "embedding",
      "numDimensions": 384,
      "similarity": "cosine"
    }
  ]
}
```

7. Click "**Create Search Index**"
8. Wait 1-2 minutes for status → **READY** ✓

---

## PART 4 — CHATBOT CODE

`personal_code.py` — Backend Logic

```
python
```

```

import os
from langchain_core.prompts import PromptTemplate
from langchain_groq import ChatGroq
from pymongo import MongoClient
from langchain_huggingface import HuggingFaceEmbeddings
import streamlit as st

# Load secrets
groq_api = st.secrets["groq_api"]
mongo_db = st.secrets["mongo_db"]
os.environ["GROQ_API_KEY"] = groq_api

# Initialize LLM
llm = ChatGroq(model="openai/gpt-oss-120b", temperature=0.7, max_tokens=1000)

# Initialize embeddings
embedding = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-

# Connect to MongoDB
client = MongoClient(mongo_db, tls=True, tlsAllowInvalidCertificates=True)
collection = client["sample_mflix"]["personalpdf"]

def get_query_results(query):
    """Vector search in MongoDB"""
    query_embedding = embedding.embed_query(query)
    pipeline = [
        {"$vectorSearch": {
            "index": "vector_index",
            "queryVector": query_embedding,
            "path": "embedding",
            "numCandidates": 384,
            "limit": 10
        }},
        {"$project": {"_id": 0, "text": 1}}
    ]
    results = collection.aggregate(pipeline)
    return [doc for doc in results]

def llm_response(query):
    """Generate response using LLM"""
    context_docs = get_query_results(query)
    context_string = " ".join([doc["text"] for doc in context_docs])
    prompt = f"""Use the following context to answer the question.

```



```
{context_string}
Question: {query}
"""

response = llm.invoke(prompt)
return response.content
```

---

## PART 5 — RUN THE APP

### Every Time You Start Working:

```
bash

# Step 1 - Activate virtual environment
cb_venv\Scripts\activate

# Step 2 - Run the app
streamlit run streamlit_runner.py
```

Your app opens at: <http://localhost:8501>

---

## PART 6 — GIT & GITHUB

### Step 1 — Install Git

- Download from [git-scm.com/download/win](https://git-scm.com/download/win)
- During install select:
  - Default editor: **VS Code**
  - Initial branch: **main**
  - PATH: **Git from command line and 3rd party**
  - SSH: **Use bundled OpenSSH**
  - HTTPS: **Use native Windows Secure Channel**
  - Line endings: **Checkout Windows-style**
- Keep all other defaults

### Step 2 — Create `.gitignore`

```
.env
.streamlit/secrets.toml
```

```
cb_venv/  
__pycache__/  
*.pyc
```

### Step 3 — Configure Git Identity

```
bash  
  
git config --global user.name "Manoj Nahak"  
git config --global user.email "your@email.com"
```

### Step 4 — Initialize Git

```
bash  
  
git init  
git add .  
git commit -m "Initial commit - Manoj RAG Chatbot"
```

### Step 5 — Create GitHub Repository

1. Go to **github.com** → Login
2. Click "+" → "New Repository"
3. Name:
4. Keep **Public**
5. Don't add README/LICENSE
6. Click "Create Repository"

### Step 6 — Push Code to GitHub

```
bash  
  
git remote add origin https://github.com/manojnahak02/manoj-rag-chatbot.git  
git branch -M main  
git push -u origin main --force
```

---

## ⚠ Common Errors & Fixes

| Error                                          | Cause                      | Fix                                                               |
|------------------------------------------------|----------------------------|-------------------------------------------------------------------|
| <code>ModuleNotFoundError</code>               | Package not installed      | <code>pip install package-name</code>                             |
| <code>StreamlitSecretNotFoundError</code>      | secrets.toml missing       | Create <code>.streamlit/secrets.toml</code>                       |
| <code>ServerSelectionTimeoutError</code>       | Wrong MongoDB URI          | Check <code>.env</code> connection string                         |
| <code>authentication failed</code>             | Wrong password             | Reset password in MongoDB Atlas                                   |
| <code>InvalidURI</code>                        | Special chars in password  | Use only letters and numbers                                      |
| <code>git not recognized</code>                | Git not installed          | Install from <a href="https://git-scm.com">git-scm.com</a>        |
| <code>rejected (fetch first)</code>            | GitHub has different files | Run <code>git pull origin main --allow-unrelated-histories</code> |
| <code>langchain_text_splitter not found</code> | Wrong import name          | Use <code>langchain_text_splitters</code> (with s)                |

## 📖 Quick Reference — Daily Commands

bash

```
# Start working
cb_venv\Scripts\activate
streamlit run streamlit_runner.py

# Save changes to GitHub
git add .
git commit -m "your message here"
git push

# Upload new PDF data
python upload_data.py
```

---

## 🌟 Future Improvements (from README)

- PDF upload support directly in UI
- Conversation memory
- Deploy on Streamlit Cloud
- Authentication system

---

## 💻 Author

**Manoj Nahak** Aspiring Data Scientist | Machine Learning Engineer GitHub:  
[github.com/manojnahak02](https://github.com/manojnahak02)